Part 1: Project Nova Management

Team Member / Role	Technical Action Description	Exact Git Terminal Command
Owner (Cuevas)	Initializes the Project Nova repository baseline configuration.	git init
Member 2 UI (Jadulco)	Syncs up with the project cloud, spawns a unique feature track, and creates the UI file.	git checkout -b feature/user-ui echo "// UI Layer" > ui.js
Member 2 UI (Jadulco)	Saves their UI progress locally and publishes the unique feature branch to GitHub.	git commit -m "Completed UI task assignment" git push origin feature/user-ui
Member 3 Database (Dizon)	Syncs up, builds an isolated database track, and saves their backend configuration.	git checkout -b feature/database-setup echo "// DB Config" > db.js
Member 3 Database (Dizon)	Saves their database progress locally and publishes their track to GitHub.	git commit -m "Completed Database task assignment" git push origin feature/database-setup
Member 1 Payment (Monteroyo)	Spawns and switches to an isolated local track for payment features.	git checkout -b feature/payment-gateway
Member 1 Payment (Monteroyo)	Stages and commits the initial payment processing shell code.	git add payment.js git commit -m "Added payment processing shell"
Owner (Cuevas)	Stages base environment configuration files directly on the primary trunk.	git add config.js
Owner (Cuevas)	Commits base changes to main before exposing the branch to team pulls.	git commit -m "Added core system configurations"
Member 1 Payment (Monteroyo)	Alters the exact same configuration blocks on their track, setting up a collision.	git commit -m "Added payment specific configurations"
Member 1 Payment (Monteroyo)	Directs a pull from the remote trunk, triggering an intentional **MERGE CONFLICT**.	git pull origin main
Member 1 Payment (Monteroyo)	Manually sweeps clashing markers out of config.js and stages the clean file.	git add config.js
Member 1 Payment (Monteroyo)	Executes the final resolution commit to patch the divergence and secure the branch.	git commit -m "Resolved merge conflict in config.js"
Owner (Cuevas)	Realigns terminal to main, pulls the latest state, and merges Member 2's UI branch.	git checkout main git pull origin main git merge origin/feature/user-ui
Owner (Cuevas)	Merges Member 3's backend database feature track directly into the master branch.	git merge origin/feature/database-setup
Owner (Cuevas)	Executes the final deployment push to upload the completely unified project to GitHub.	git push origin main

Part 2: TROUBLESHOOTING & EMERGENCY FIXES

Provide the command(s) needed to fix the issue and save the day.

Case	Scenario / Problem	Exact Git Recovery Command(s)
1	**Unstage Wrong File:** Accidentally staged a sensitive file (.env) using git add ..	git restore --staged .env
2	**Fix Typos:** Misspelled a word in the last commit message before pushing.	git commit --amend -m "Fixed authentication typo"
3	**Include Missed File:** Forgot to add styles.css to the commit you just made.	git add styles.css git commit --amend --no-edit
4	**Discard Working Changes:** Local edits to index.html broke the file; want to reset it.	git restore index.html
5	**Commit on Wrong Branch:** Accidental 3 commits on main instead of a feature branch.	git branch new-feature git reset HEAD~3 –hard git checkout new-feature
6	**Save Detached HEAD:** Committed changes while exploring history; need to save them.	git switch -c recovery-branch
7	**Undo Pushed Bug:** Pushed a bad commit to GitHub and need to safely reverse it.	git revert <buggy-commit-hash>
8	**Recover Deleted Branch:** Restoring an unmerged branch deleted by accident.	git branch <deleted-branch-name> <commit-hash>
9	**Sync OS File Deletions:** Staging multiple file deletions done via File Explorer.	git add -u
10	**Delete Remote Branch:** Accidentally pushed a junk branch (testing-junk) to GitHub.	git push origin --delete testing-junk